第四章

自顶向下的语法分析

重点：自顶向下分析的基本思想，预测分析器总体结构，预测分析表的构造，递归下降分析法基本思想，简单算术表达式的递归下降分析器。

难点：FIRST 和 FOLLOW 集的求法，对它们的理解以及在构造LL(1)分析表时的使用。递归子程序法中如何体现分析的结果。

第4章 自顶向下的语法分析

4.1 语法分析概述

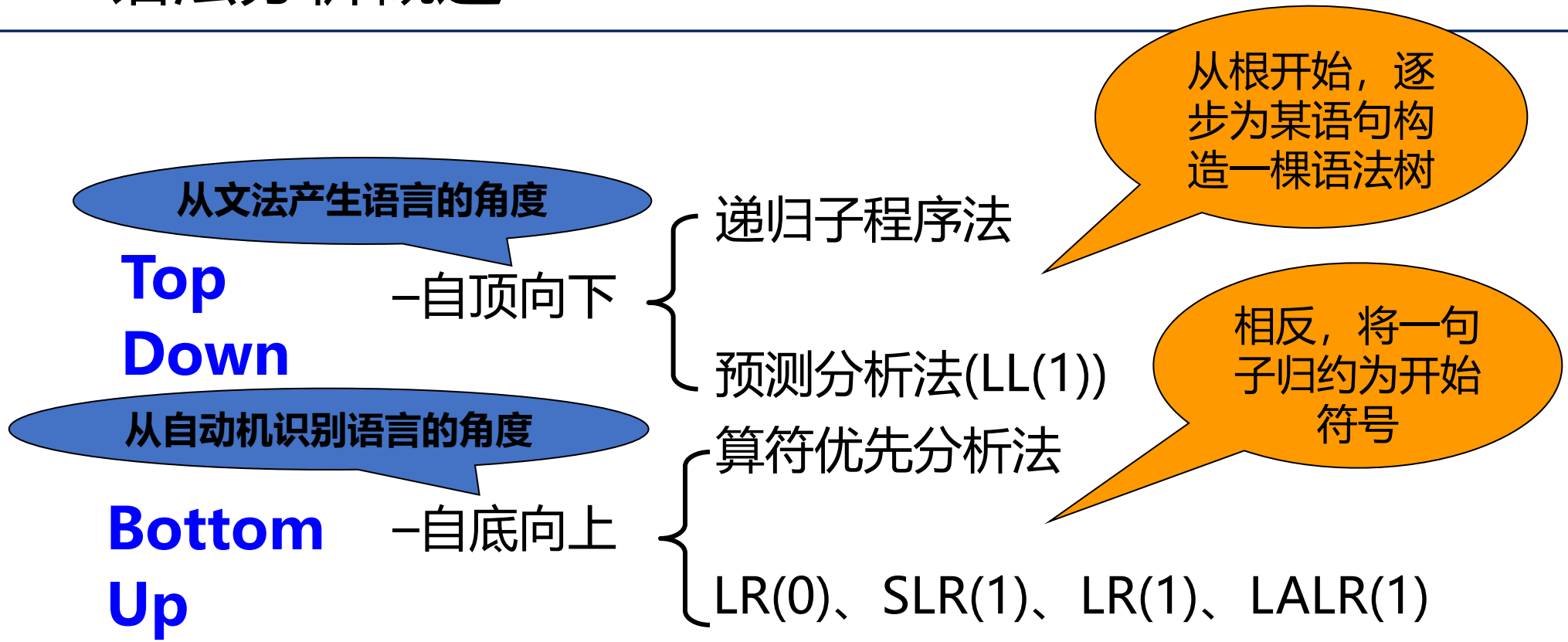
4.2 自顶向下的语法分析面临的问题

4.3 预测分析法

4.4 递归下降分析法

4.5 本章小结

4.1 语法分析概述



问题：解决确定性问题！

假定文法是压缩的：即删除了单位产生式和无用产生式。

4.2.3 LL(1)文法

如果G的任意两个具有相同左部的产生式$A\rightarrow\alpha|\beta$满足下列条件：

1. 如果α、β均不能推导出ε，则$\text{FIRST}(\alpha)\cap\text{FIRST}(\beta)=\varnothing$；
2. α和β至多有一个能推导出ε；
3. 如果$\beta\overset{*}{\Rightarrow}\varepsilon$，且$\text{FIRST}(\alpha)\cap\text{FOLLOW}(A)=\varnothing$

则称G为*LL(1)文法*

第一个L代表从左向右扫描输入符号串，第二个L代表产生最左推导，1代表在分析过程中执行每步推导都要向前查看一个输入符号

例　表达式文法的语法符号的FIRST 集

FIRST(F)={ (, id}

FIRST(T)=FIRST(F)={ (, id}

FIRST(E)=FIRST(T)={ (, id}

FIRST(E')={+, ε}

FIRST(T')={*, ε}

FIRST(+)={+}, FIRST(*)={*}

FIRST(()={ (}

FIRST())={) }

FIRST(id)={id}

E→TE'

E'→+TE' |ε

T→FT'

T'→*FT'|ε

F→(E)|id

例 表达式文法的语法变量的 FOLLOW 集

FOLLOW(S) := {#}，#为句子的结束符;

若$A \to \alpha B \beta \in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)–{ε};

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FOLLOW(E) = { # ,) }

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

例 表达式文法的语法变量的 FOLLOW 集

4．若$A \to \alpha B\beta \in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)−{ε};

5．若$A \to \alpha B$或$A \to \alpha B\beta \in P$，且$\beta \Rightarrow^{*} \varepsilon$，$A \neq B$，则

FOLLOW(B) := FOLLOW(B)∪FOLLOW(A);

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FOLLOW(E) = { #,) }

FOLLOW(E')= FOLLOW(E) = { #,) }

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

例 表达式文法的语法变量的 FOLLOW 集

4．若$A \to \alpha B \beta \in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)−{ε};

5．若$A \to \alpha B$或$A \to \alpha B\beta \in P$，且$\beta \Rightarrow^{*} \varepsilon$，$A \neq B$，则

FOLLOW(B) := FOLLOW(B)∪FOLLOW(A);

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FOLLOW(E) = { #,) }

FOLLOW(E')= FOLLOW(E) = { #,) }

FOLLOW(T) = {FIRST(E')-{ε}}∪FOLLOW(E)∪FOLLOW(E')= {+,),#}

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

例 表达式文法的语法变量的 FOLLOW 集

4．若$A \to \alpha B \beta \in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)−{ε};

5．若$A \to \alpha B$或$A \to \alpha B \beta \in P$，且$\beta \Rightarrow^{*} \varepsilon$，$A \neq B$，则

FOLLOW(B) := FOLLOW(B)∪FOLLOW(A);

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

FOLLOW(E) = { #,) }

FOLLOW(E')= FOLLOW(E) = { #,) }

FOLLOW(T) = {FIRST(E')-{ε}}∪FOLLOW(E)∪FOLLOW(E')= {+,),#}

例 表达式文法的语法变量的 FOLLOW 集

4．若$A \to \alpha B\beta \in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)−{ε};

5．若$A \to \alpha B$或$A \to \alpha B\beta \in P$，且$\beta \overset{*}{\Rightarrow} \varepsilon$，$A \neq B$，则

FOLLOW(B) := FOLLOW(B)∪FOLLOW(A);

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FOLLOW(E) = { #,) }

FOLLOW(E')= FOLLOW(E) = { #,) }

FOLLOW(T) = {FIRST(E')-{ε}}∪FOLLOW(E)∪FOLLOW(E')= {+,),#}

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

例 表达式文法的语法变量的 FOLLOW 集

4．若$A \to \alpha B \beta \in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)−{ε};

5．若$A \to \alpha B$或$A \to \alpha B\beta \in P$，且$\beta \overset{*}{\Rightarrow} \varepsilon$，$A \neq B$，则

FOLLOW(B) := FOLLOW(B)∪FOLLOW(A);

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

FOLLOW(E) = { #,) }

FOLLOW(E')= FOLLOW(E) = { #,) }

FOLLOW(T) = {FIRST(E')-{ε}}∪FOLLOW(E)∪FOLLOW(E')= {+,),#}

FOLLOW(T')= FOLLOW(T)= {+,),#}

例 表达式文法的语法变量的 FOLLOW 集

4．若$A \to \alpha B \beta \in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)−{ε};

5．若$A \to \alpha B$或$A \to \alpha B\beta \in P$，且$\beta \stackrel{*}{\Rightarrow} \varepsilon$，$A \neq B$，则

FOLLOW(B) := FOLLOW(B)∪FOLLOW(A);

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FOLLOW(E) = { #,) }

FOLLOW(E')= FOLLOW(E) = { #,) }

FOLLOW(T) = {FIRST(E')-{ε}}∪FOLLOW(E)∪FOLLOW(E')= {+,),#}

FOLLOW(T')= FOLLOW(T)= {+,),#}

FOLLOW(F) = {FIRST(T') -{ε}}∪FOLLOW(T)∪FOLLOW(T') ={*,+,),#}

例 表达式文法的语法变量的 FOLLOW 集

4．若$A\to\alpha B\beta\in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)−{ε};

5．若$A\to\alpha B$或$A\to\alpha B\beta\in P$，且$\beta \overset{*}{\Rightarrow}\varepsilon$，$A\neq B$，则

FOLLOW(B) := FOLLOW(B)∪FOLLOW(A);

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FOLLOW(E) = { #,) }

FOLLOW(E')= FOLLOW(E) = { #,) }

FOLLOW(T) = {FIRST(E')-{ε}}∪FOLLOW(E)∪FOLLOW(E')= {+,),#}

FOLLOW(T')= FOLLOW(T)= {+,),#}

FOLLOW(F) = {FIRST(T') -{ε}}∪FOLLOW(T)∪FOLLOW(T') ={*,+,),#}

例 表达式文法的语法变量的 FOLLOW 集

4．若$A\to\alpha B\beta\in P$，则

FOLLOW(B):=FOLLOW(B)∪FIRST(β)−{ε};

5．若$A\to\alpha B$或$A\to\alpha B\beta\in P$，且$\beta\stackrel{*}{\Rightarrow}\varepsilon$，$A\neq B$，则

FOLLOW(B) := FOLLOW(B)∪FOLLOW(A);

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

E→TE'　E'→+TE'|ε

T→FT'　T'→*FT'|ε

F→(E)|id

FOLLOW(E) = { #,) }

FOLLOW(E')= FOLLOW(E) = { #,) }

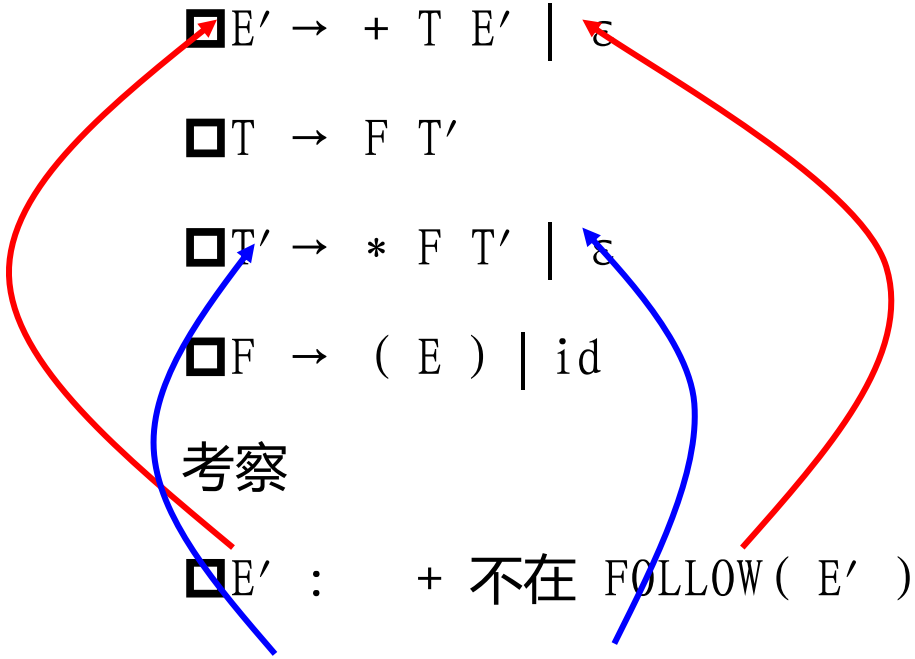
FOLLOW(T) = {FIRST(E')-{ε}}∪FOLLOW(E)∪FOLLOW(E')= {+,),#}

FOLLOW(T')= FOLLOW(T)= {+,),#}

FOLLOW(F) = {FIRST(T') -{ε}}∪FOLLOW(T)∪FOLLOW(T') ={*,+,),#}

表达式文法是 LL(1) 文法

- E → T E′
- E′ → + T E′ | ε
- T → F T′
- T′ → * F T′ | ε
- F → (E) | id



考察

- E′ : + 不在 FOLLOW(E′) = {), # }
- T′ : * 不在 FOLLOW(T′) = { +,), # }
- F: (和 id 不同

如果G的任意两个具有相同左部的产生式$A\to\alpha|\beta$满足下列条件：

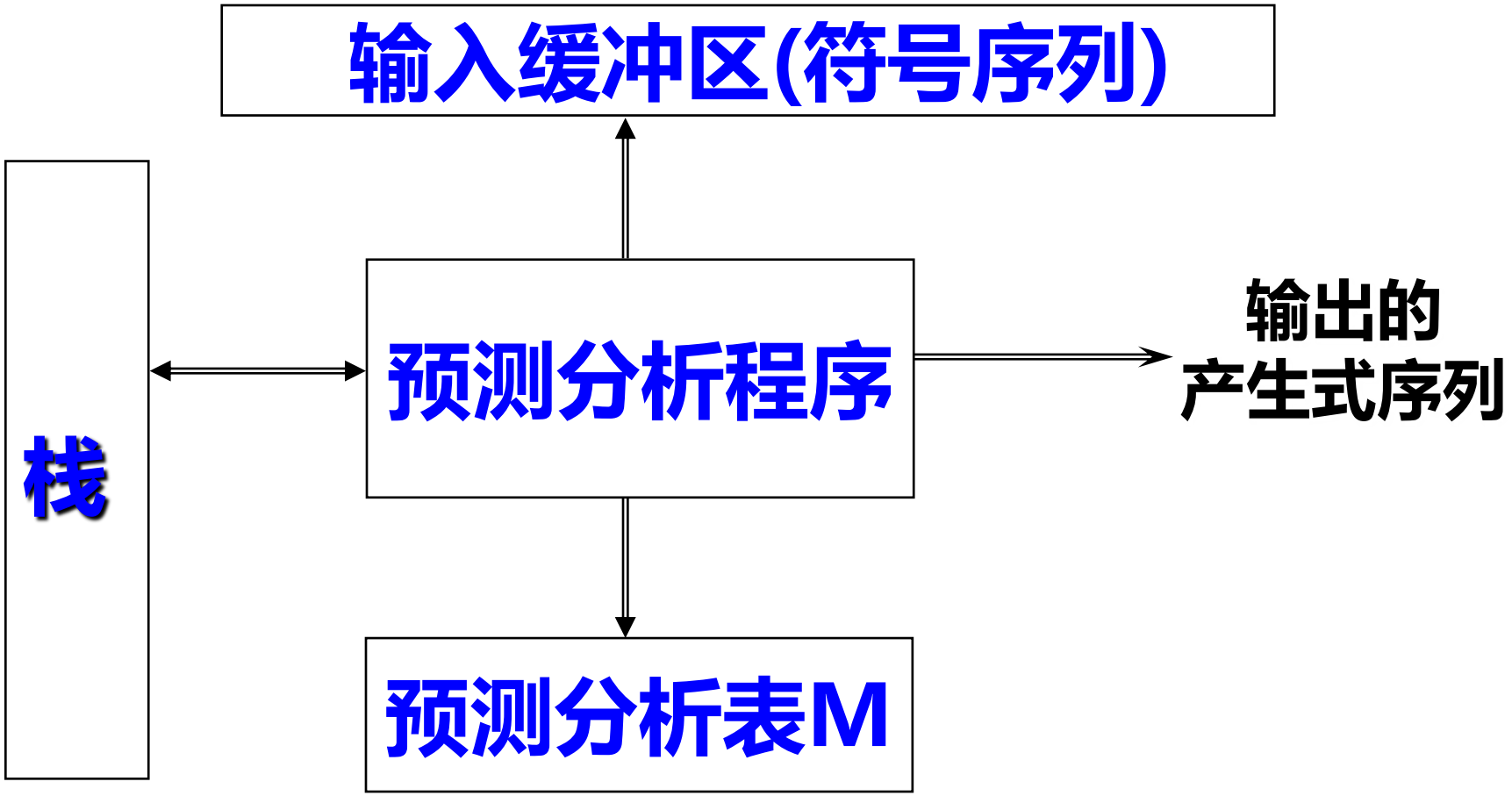
1. 如果α、β均不能推导出ε，则$FIRST(\alpha)\cap FIRST(\beta)=\varnothing$；
2. α和β至多有一个能推导出ε；
3. 如果$\beta\overset{*}{\Rightarrow}\varepsilon$，且$FIRST(\alpha)\cap FOLLOW(A)=\varnothing$

则称G为*LL(1)*文法

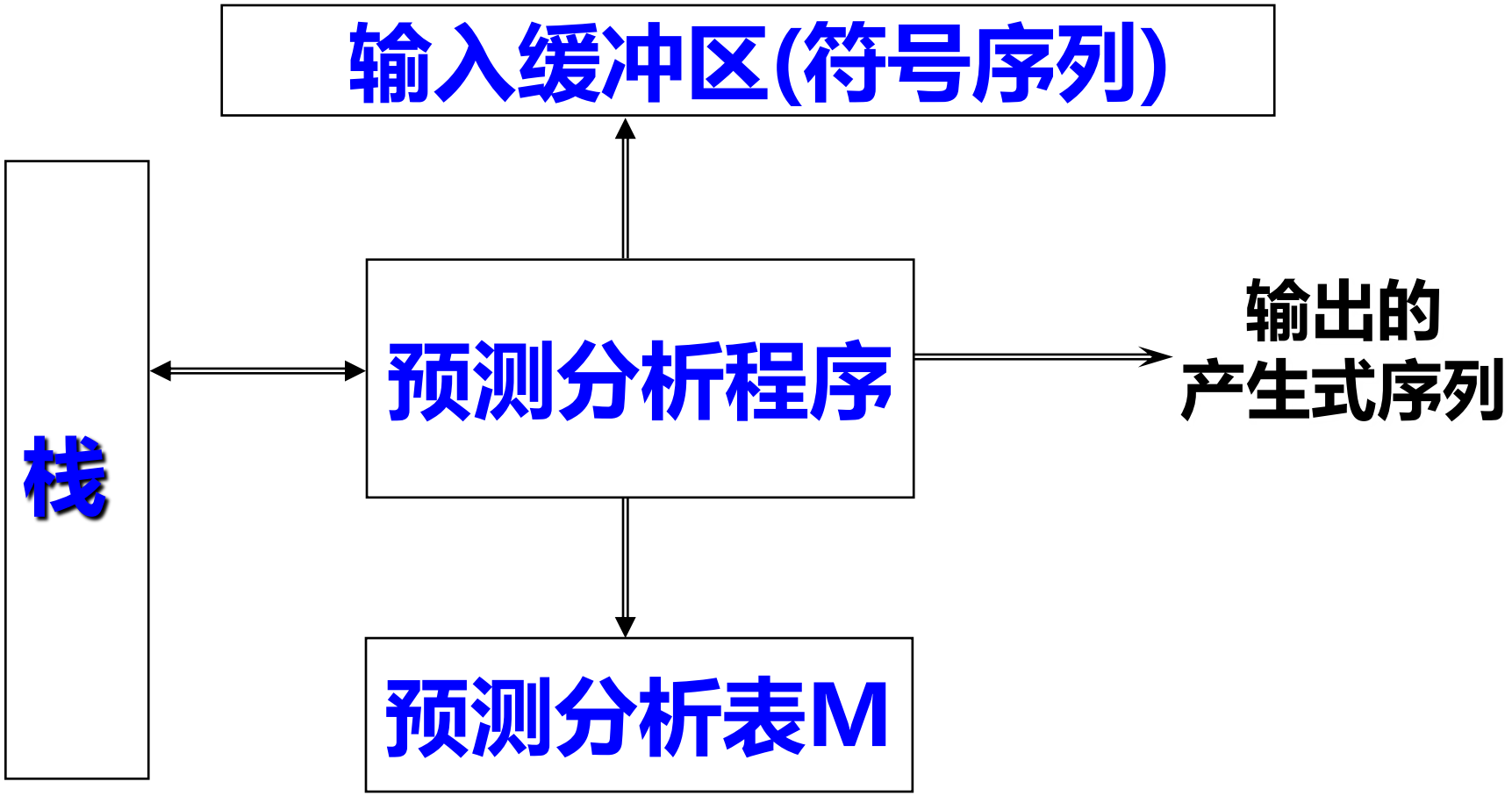
4.3 预测分析法

- 系统维持一个分析表和一个分析栈，根据当前扫描到的符号，选择当前语法变量（处于栈顶）的候选式进行推导——希望找到相应输入符号串的最左推导。
- 一个通用的控制算法
- 一个分析栈，#为栈底符号
- 一个输入缓冲区，#为输入串结束符
- 一个统一形式的分析表M
  - 不同语言使用内容不同的分析表

4.3.1 预测分析器的构成

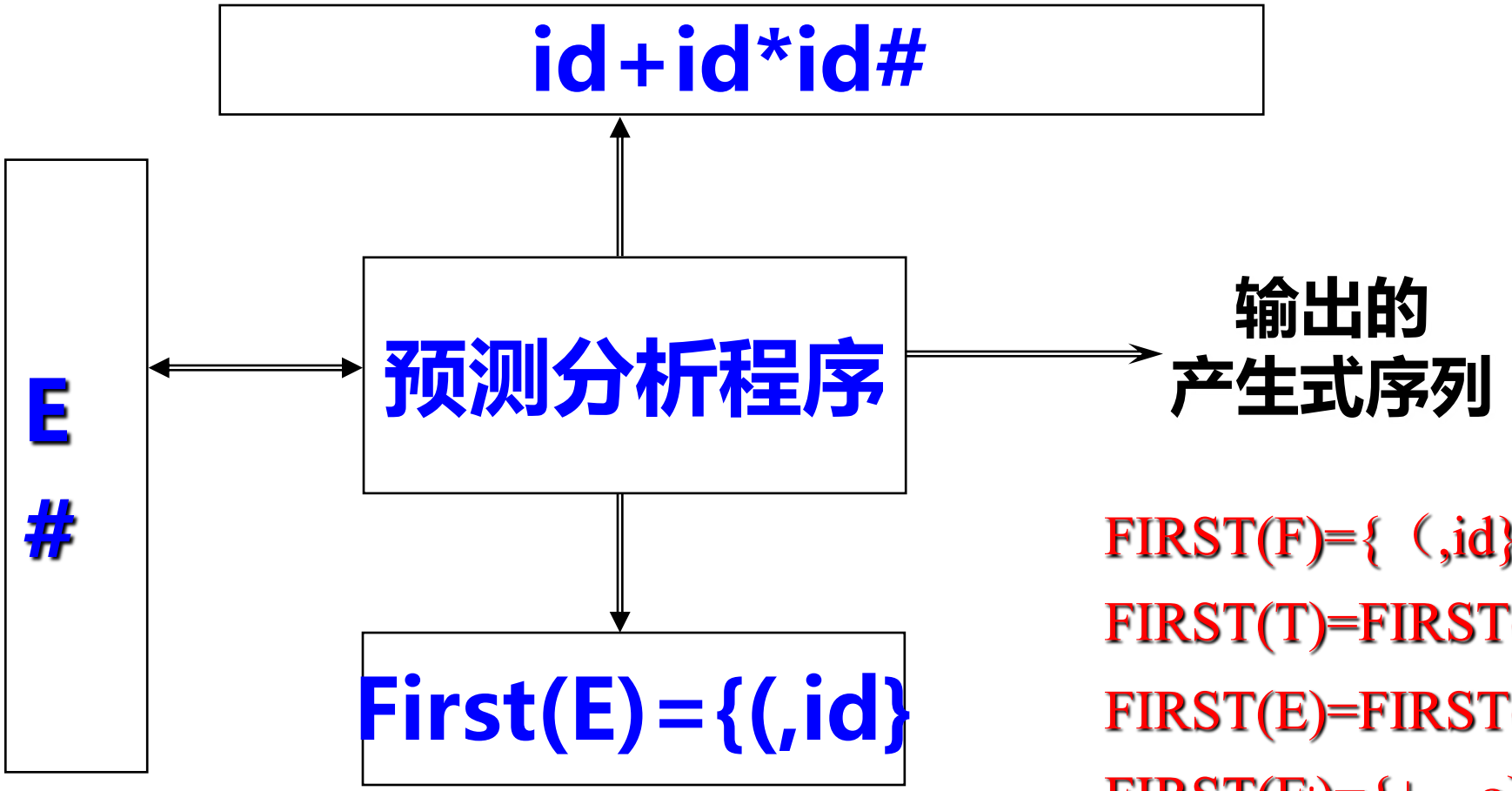


4.3.1 预测分析器的构成



4.3.1 预测分析器的构成

$E \to TE'$　$E' \to +TE' \mid \varepsilon$　$T \to FT'$　$T' \to *FT' \mid \varepsilon$　$F \to (E) \mid id$



FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

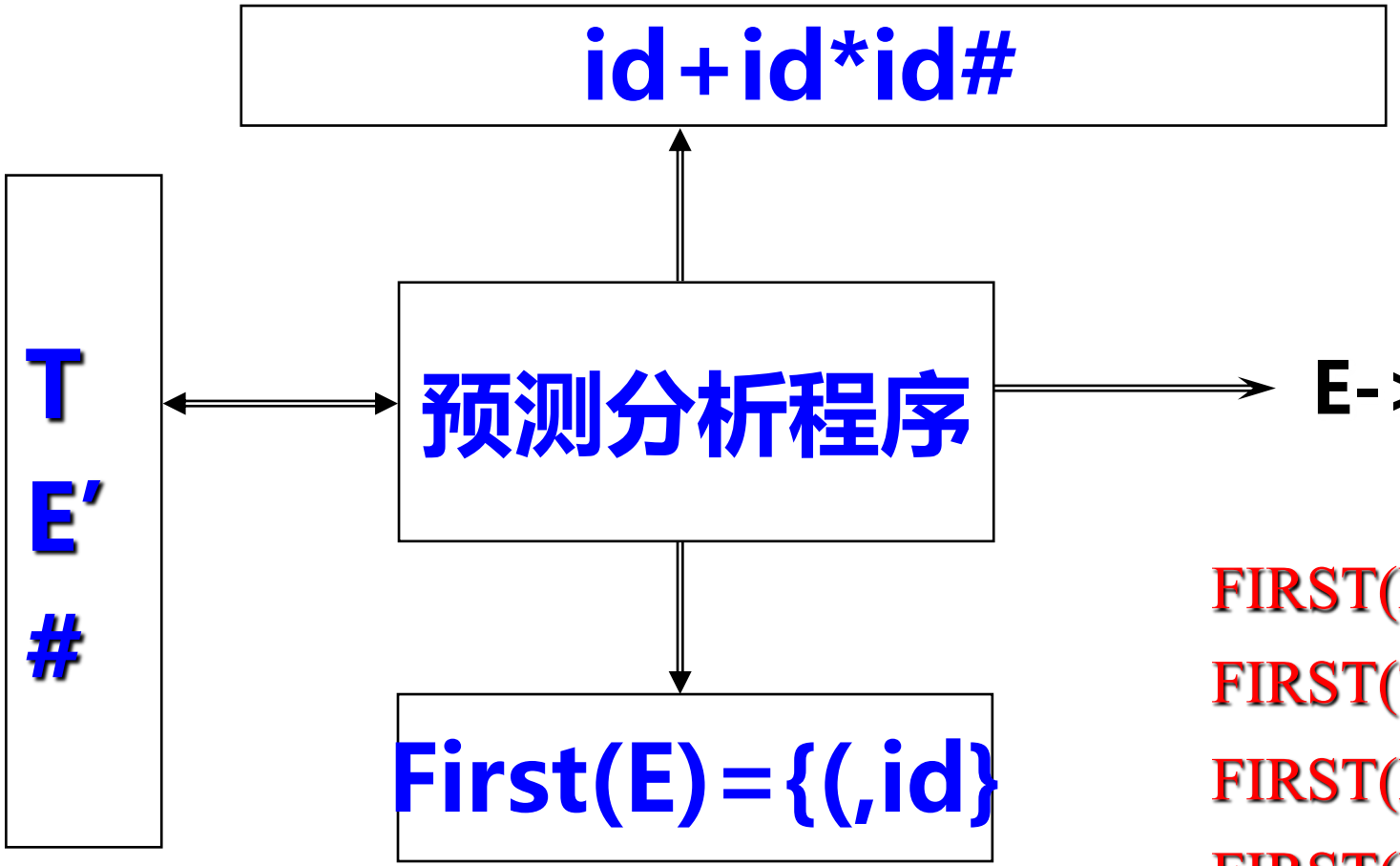
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

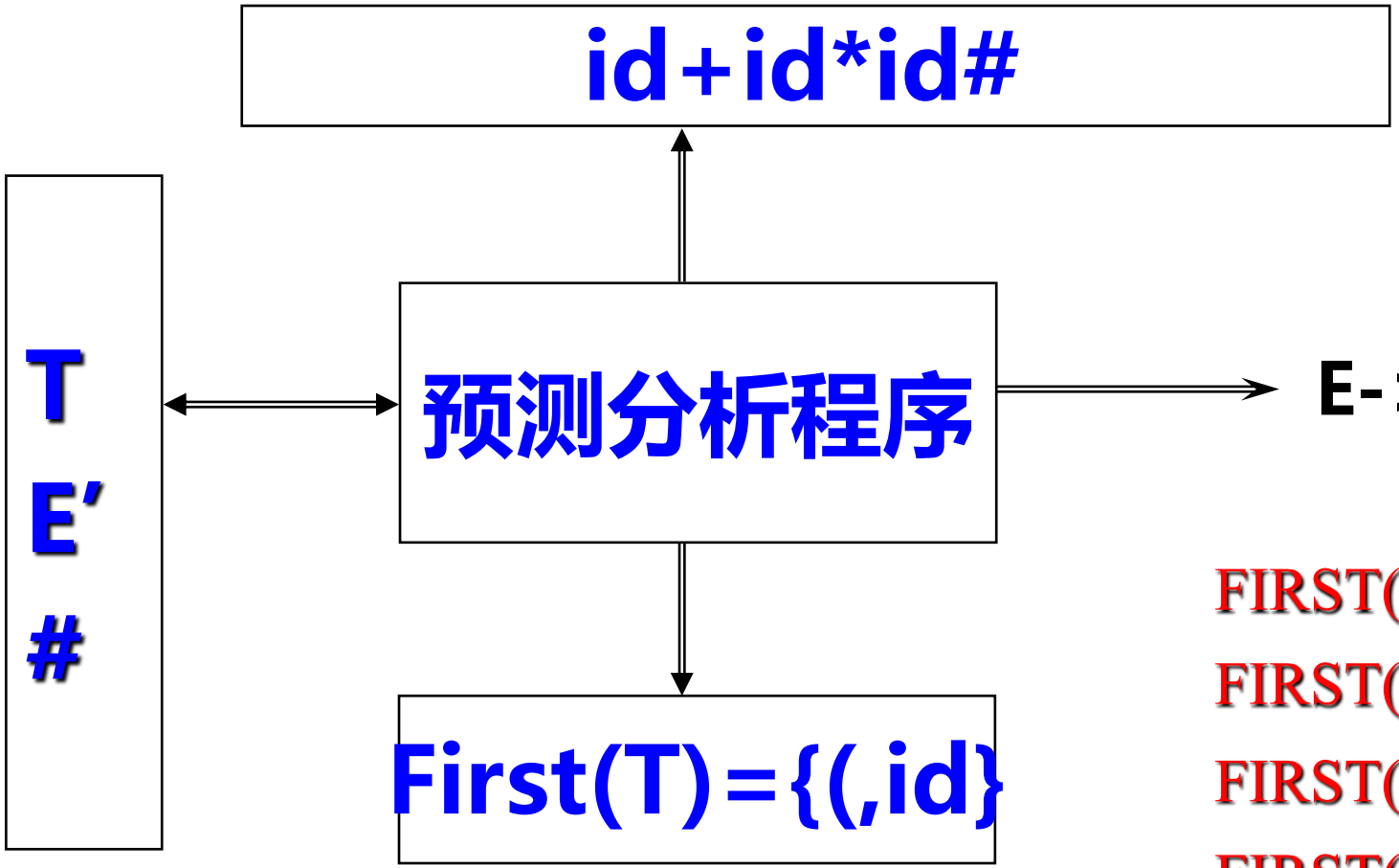
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

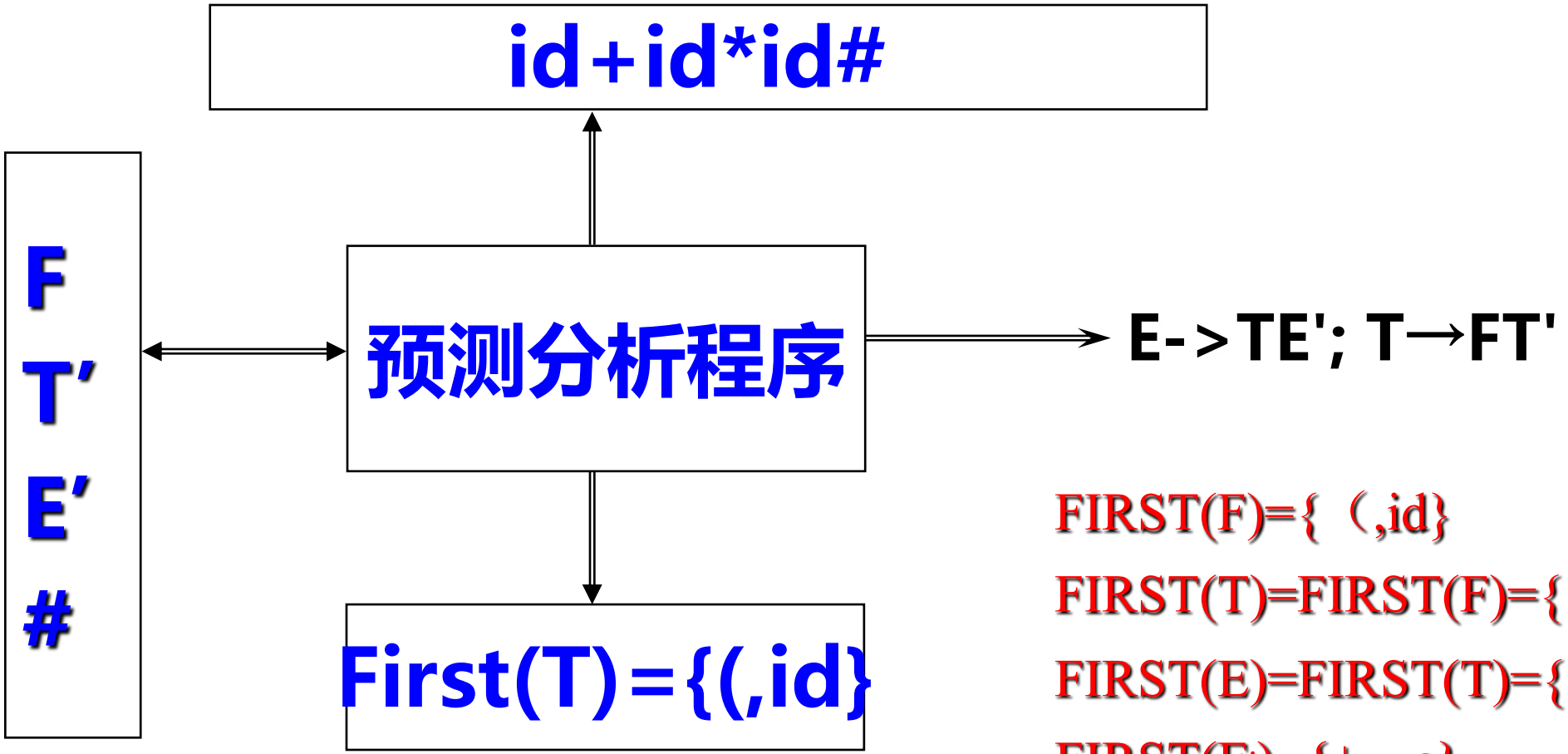
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

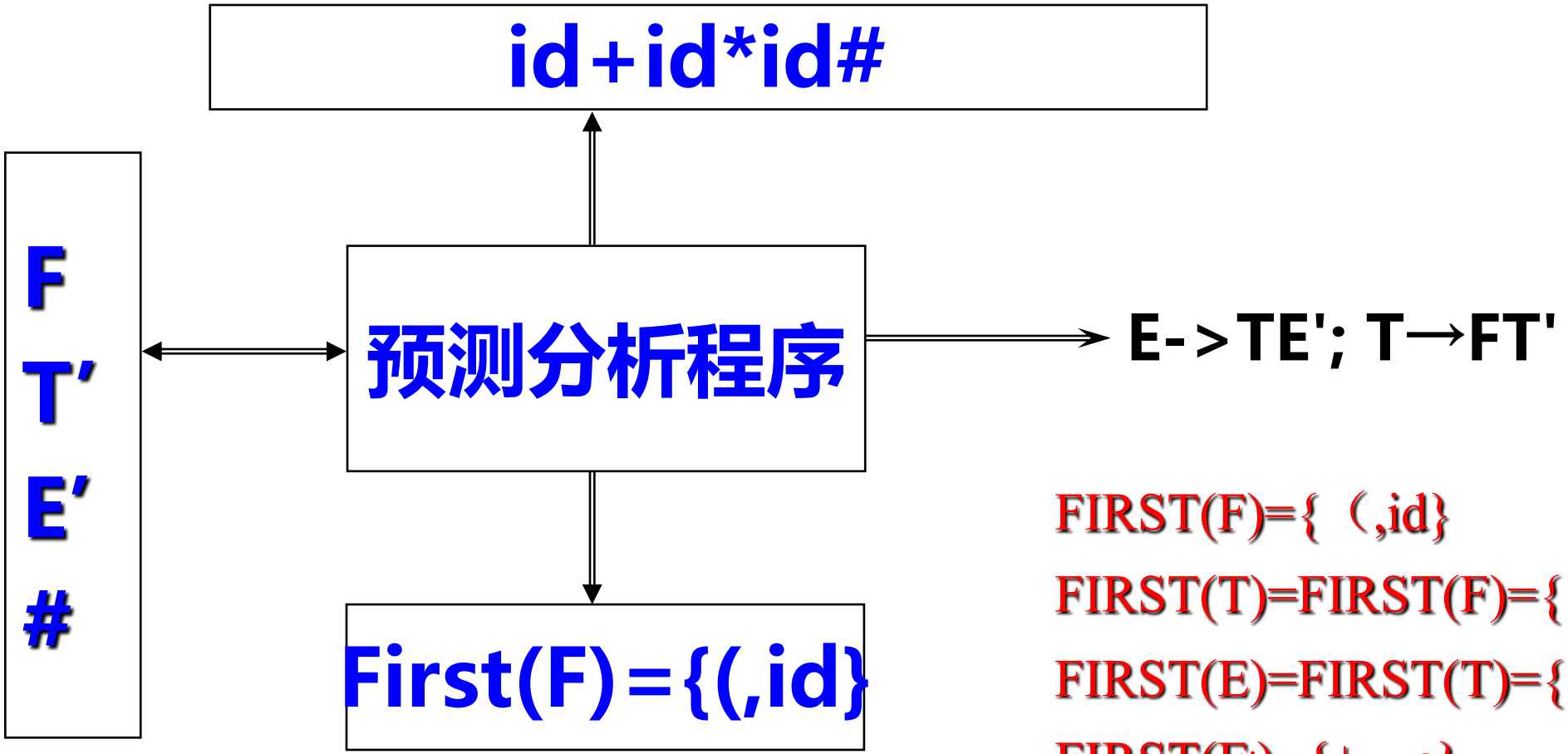
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

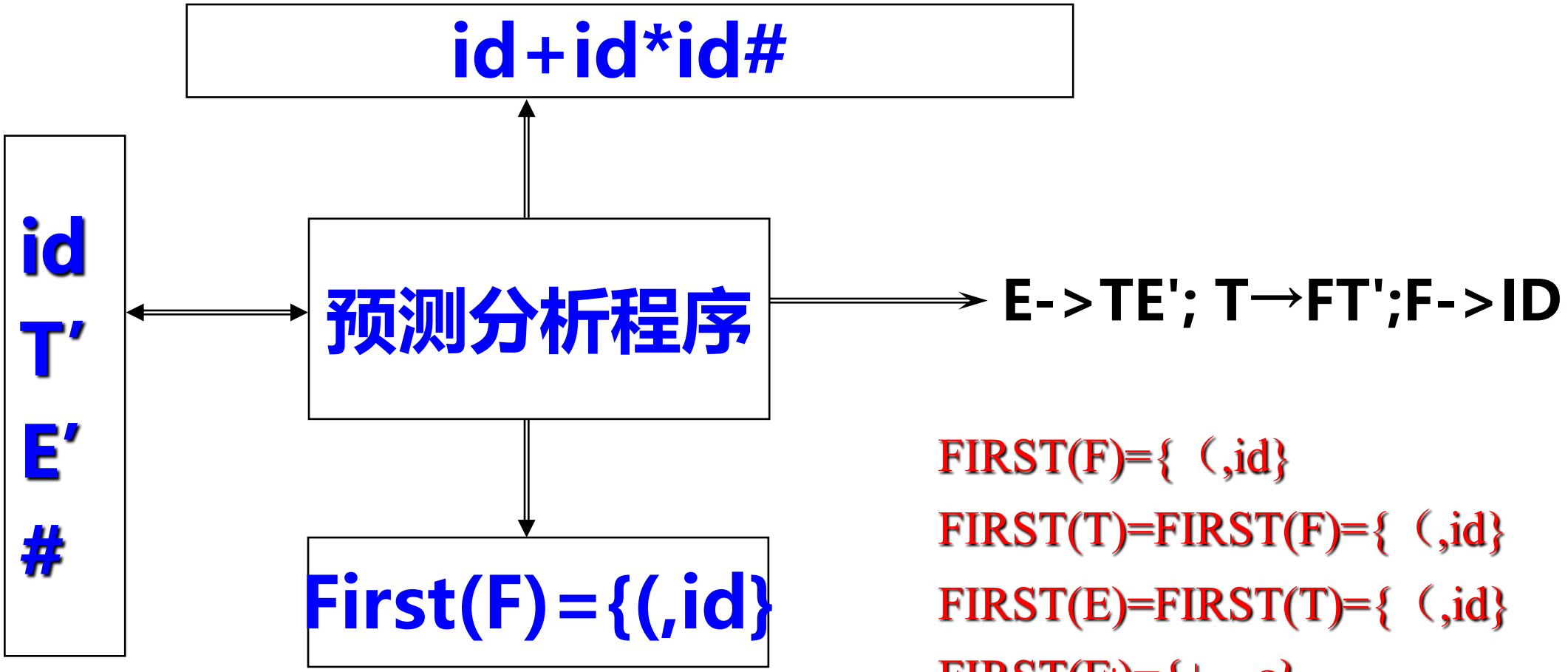
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

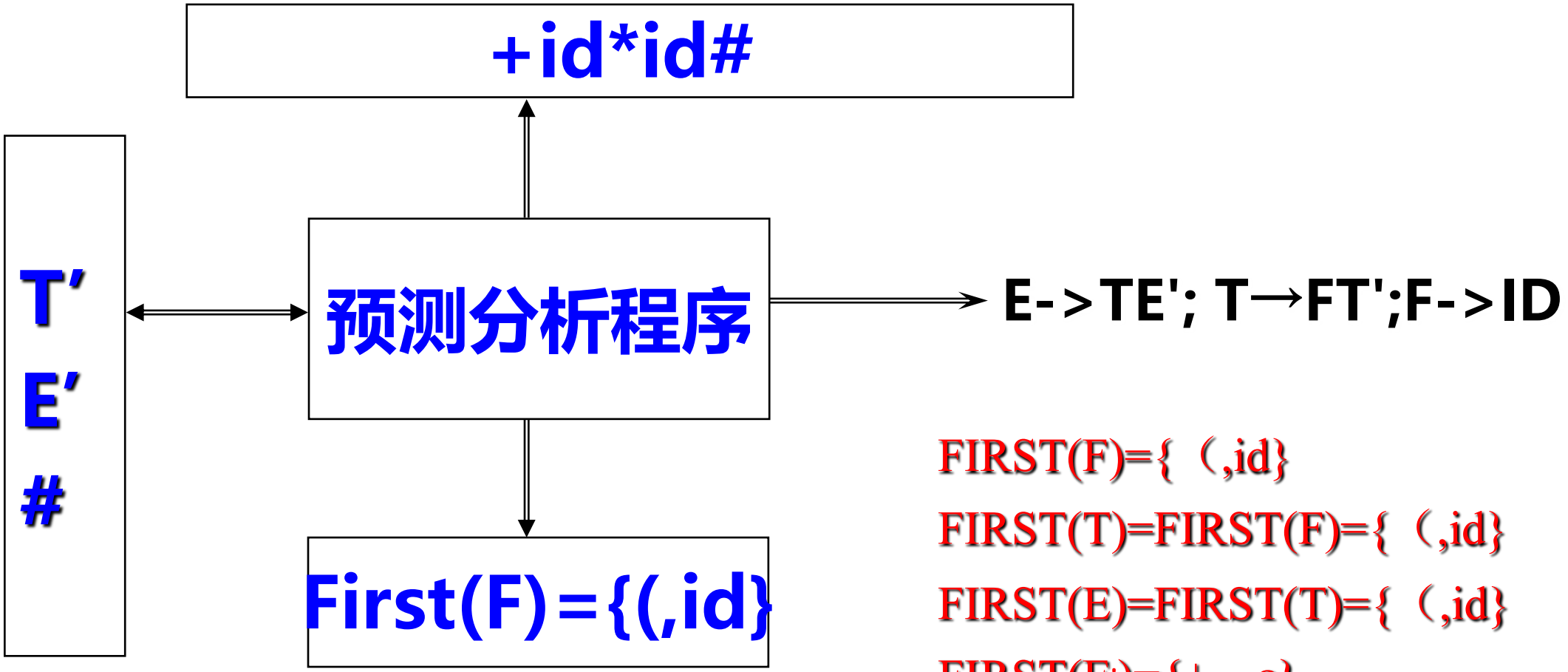
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

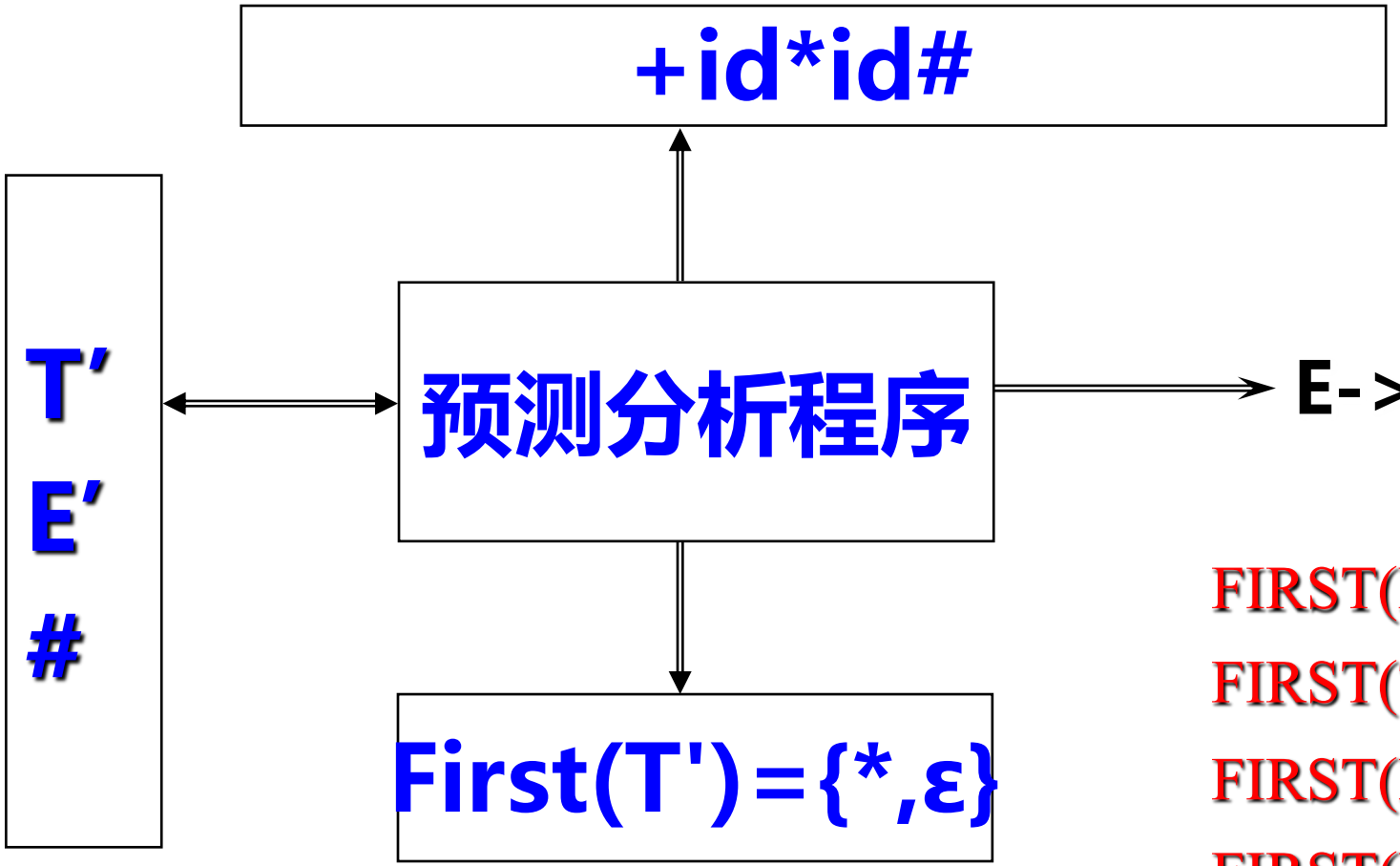
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



E->TE'; T→FT';F->ID

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

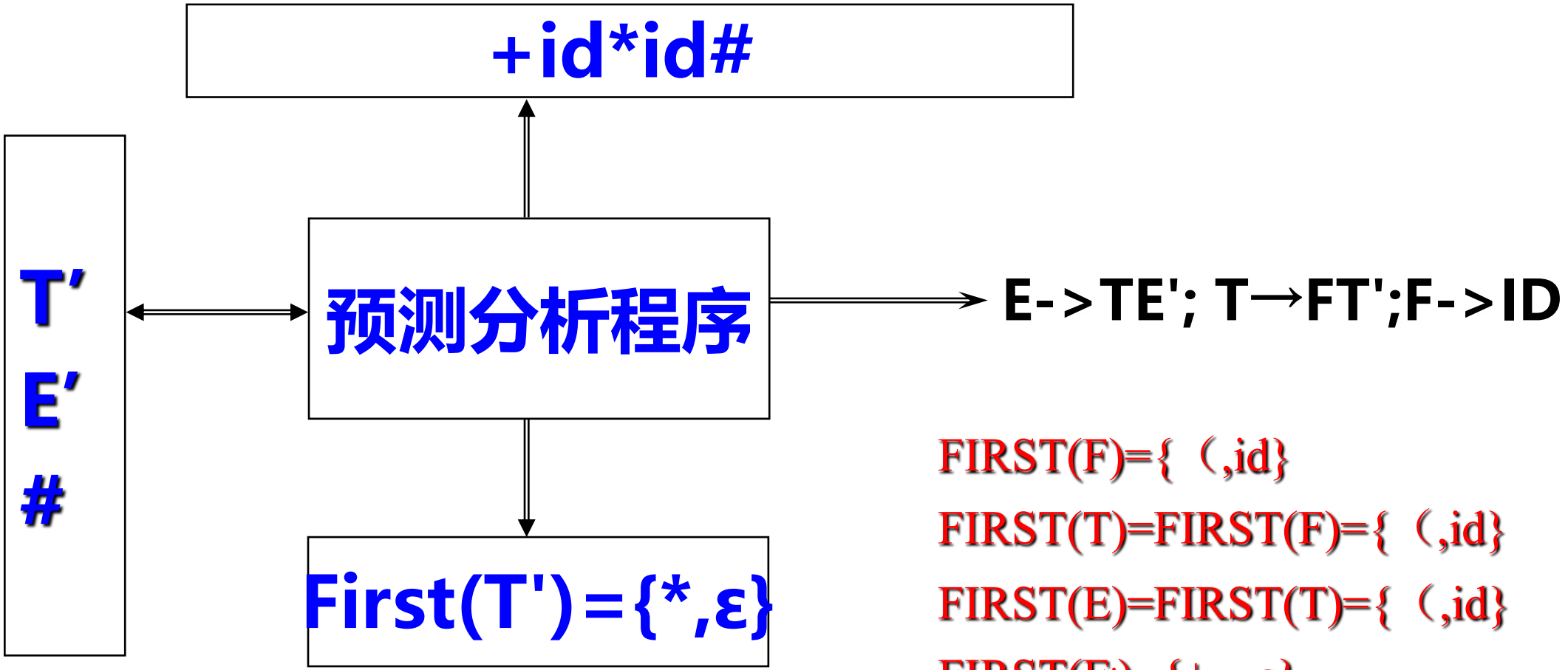
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



FOLLOW(T')={ +,), # }

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

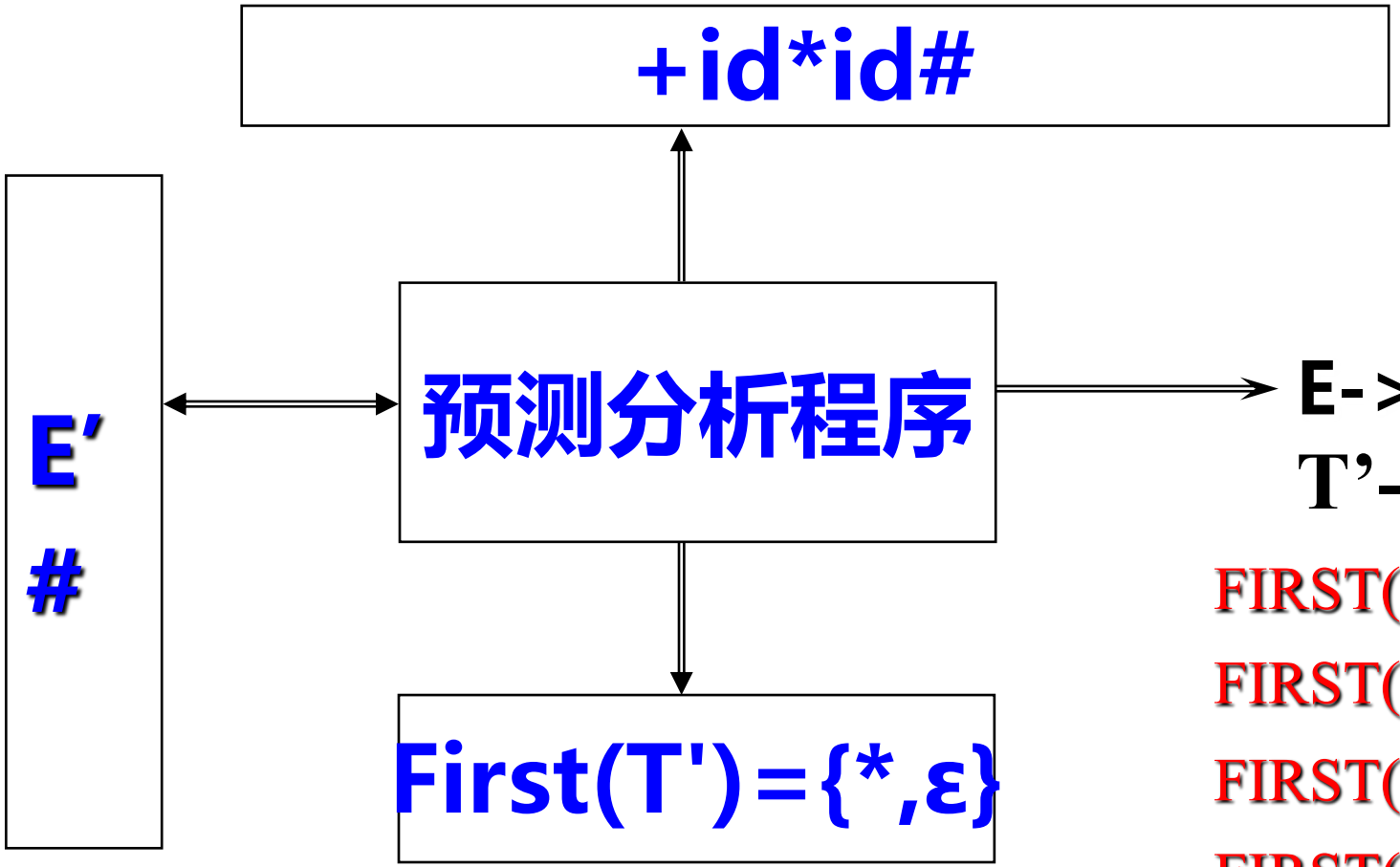
FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

4.3.1 预测分析器的构成

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id



E->TE'; T→FT';F->ID;
T’->ε

FOLLOW(T')={ +,), # }

FIRST(F)={（,id}

FIRST(T)=FIRST(F)={（,id}

FIRST(E)=FIRST(T)={（,id}

FIRST(E')={+，ε}

FIRST(T')={*,ε}

系统的执行与特点

- 在系统启动时，输入指针指向输入串的第一个字符，分析栈中存放着栈底符号#和文法的开始符号。
- 根据栈顶符号A和读入的符号a，查看分析表M, 以决定相应的动作。
- 优点：
  1）效率高
  2）便于维护、自动生成
- 关键——分析表M的构造

预测分析程序的总控程序

算法4.5 预测分析程序的总控程序。

输入：输入串w和文法$G=(V, T, P, S)$的分析表M;

输出：如果w属于$L(G)$，则输出w的最左推导，否则报告错误;

步骤：

1．将栈底符号#和文法开始符号S压入栈中;

2．repeat

3．　　X:=当前栈顶符号;

4．　　a:=当前输入符号;

5．　　if $X \in T \cup \{\#\}$ then

6．　　if $X=a$ then

7．　　{if $X \neq \#$ then begin

8．　　将X弹出栈;

9．　　前移输入指针

10．　　end}

预测分析程序的总控程序

```
11.                         else error
12.             else
13.                         if M[X, a]=Y1Y2…Yk then begin
14.                         将X弹出栈;
15.                         依次将Yk，…，Y2，Y1压入栈;
16.                         输出产生式X→Y1Y2…Yk
17.                         end
18.                         else error
19. until X=#
```

4.3.2 预测分析表的构造算法

算法4.6 预测分析表($LL(1)$分析表)的构造算法。

输入：文法G;

输出：分析表M;

步骤：

1．对G中的任意一个产生式$A\rightarrow\alpha$, 执行第2步和第3步;

2． for $\forall a\in \text{FIRST}(\alpha)$, 将$A\rightarrow\alpha$填入$M[A, a]$;

3． if $\varepsilon\in \text{FIRST}(\alpha)$ then

$\forall a\in \text{FOLLOW}(A)$,将$A\rightarrow\alpha$填入$M[A, a]$;

if $\varepsilon\in \text{FIRST}(\alpha)\&\#\in \text{FOLLOW}(A)$ then

将$A\rightarrow\alpha$填入$M[A, \#]$;

4．将所有无定义的$M[A, b]$标上出错标志。

4.3.2 预测分析表的构造算法

E→TE‘　E’→+TE’|ε　T→FT‘　T’→*FT’|ε　F→(E)|id

算法4.6 预测分析表(*LL*(1)分析表)的构造算法。

输入：文法*G*;

输出：分析表*M*;

步骤：

1．对*G*中的任意一个产生式$A\to\alpha$, 执行第2步和第3步;

2． for $\forall b\in$FIRST(α), 将$A\to\alpha$填入$M[A, b]$;

3． if $\varepsilon\in$FIRST(α) then

$\forall b\in$FOLLOW(A),将$A\to\alpha$填入$M[A, b]$;

if $\varepsilon\in$FIRST(α)&#$\in$FOLLOW(A) then

将$A\to\alpha$填入$M[A, \#]$;

4．将所有无定义的$M[A, t]$标上出错标志。

FOLLOW(E')={), # }

FOLLOW(T')={ +,), # }

FIRST(TE')={(,id}　　FIRST(+TE')={+}

FIRST(FT')={(,id}　　FIRST(*FT')={*}

FIRST((E))={(}　　FIRST(id)={id}

非终结符	输入符号					
	id	+	*	(	)	#
E	→TE′			→TE′		
E’		→+TE′			→ε	→ε
T	→FT′			→FT′		
T’		→ε	→*FT′		→ε	→ε
F	→id			→(E)		

对输入串id+id*id进行分析的过程

（请大家在纸上同时画出语法树）

栈	输入缓冲区	输出
#E	id+id*id#	
#E′T	id+id*id#	E→TE′
#E′T′F	id+id*id#	T→FT′
#E′T′id	id+id*id#	F→id
#E′T′	+id*id#	
#E′	+id*id#	T′→ ε
#E′T+	+id*id#	E′→+TE′
#E′T	id*id#	

非终结符	输入符号					
	id	+	*	(	)	#
E	→TE′			→TE′		
E’		→+TE′			→ε	→ε
T	→FT′			→FT′		
T’		→ε	→*FT′		→ε	→ε
F	→id			→(E)		

对输入串id+id*id进行分析的过程

#E′T	id*id#	
#E′T′F	id*id#	T→FT′
#E′T′id	id*id#	F→id
#E′T′	*id#	
#E′T′F*	*id#	T′→*FT′
#E′T′F	id#	
#E′T′id	id#	F→id
#E′T′	#	
#E ‘	#	T′→ ε
#	#	E′→ ε

非终结符	输入符号					
	id	+	*	(	)	#
E	→TE′			→TE′		
E’		→+TE′			→ε	→ε
T	→FT′			→FT′		
T’		→ε	→*FT′		→ε	→ε
F	→id			→(E)		

输出的产生式序列形成了最左推导对应的分析树

预测分析法的实现步骤

1. 构造文法
2. 改造文法：消除二义性、消除左递归、提取左因子
3. 求每个候选式的FIRST集和变量的FOLLOW集
4. 检查是不是 LL(1) 文法

   若不是 LL(1),说明文法的复杂性超过自顶向下方法的分析能力，需要附加新的“信息”
5. 构造预测分析表
6. 实现预测分析器

4.3.3 预测分析中错误的处理

- 对语法变量A，如果M[A,a]无定义，并且a属于FOLLOW(A)，则增加M[A,a]为“同步点”(synch), 同步记号选择方法如下：
  - 把FOLLOW(*A*)的所有符号放入语法变量*A*的同步记号集合中。
  - 把A后高层结构的开始符号加到低层结构的同步记号集合中。
  - 把FIRST(*A*)的符号加入*A*的同步记号集合。
  - 如果语法变量可以产生空串，若出错时栈顶是这样的语法变量，则可以使用产生空串的产生式。
  - 如果符号在栈顶而不能匹配，则弹出此符号。

单选题 10分

设置

在编译过程中，如果遇到错误应该( )。

A 对错误在局部范围内进行纠正，继续向下分析

B 当发现错误时，跳过错误所在的语法单位继续分析下去

C 当发现错误时立即停止编译，待用户改正错误后再继续编译

4.4 递归下降分析法 — 一个设想

E→TE'

E'→+TE’|ε

T→FT'

T'→*FT’|ε

F→(E)|id

1. 对应每个变量设置一个处理子程序：

$$A \rightarrow X_1 X_2 \ldots X_k \ldots X_n$$

(1) 当遇到X_k是终极符号时直接进行匹配;

(2) 当遇到X_k是语法变量时就调用X对应的处理子程序.

2. 要求处理子程序是可以递归调用的

4.4.1 递归下降分析法的基本思想

例4.14 对于产生式$E' \rightarrow +TE'$，与E'对应的子程序可以按如下方式来编写：

```
procedure E'
  begin
     match('+');
     T;                    /*调用识别T的过程*/
     E'                    /*调用识别E'的过程*/
  end;
```

4.4.1 递归下降分析法的基本思想

其中，服务子程序*match*用来匹配当前的输入记号，其代码为：

```
procedure match(t:token);
  begin
     if lookhead=t  then
          lookhead:=nexttoken;
     else error              /*调用出错处理程序*/
  end;
```

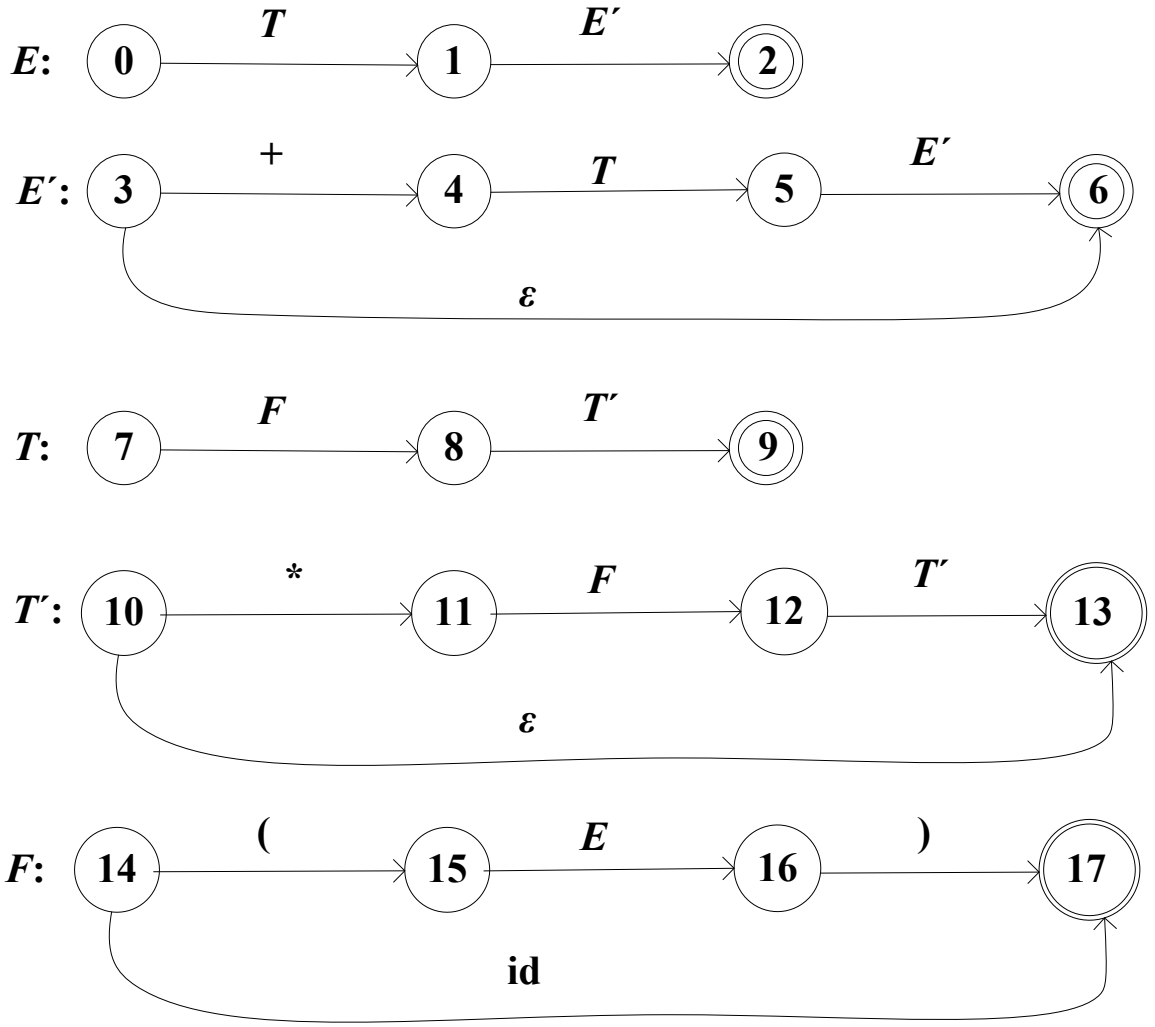
4.4.2 语法图和递归子程序法

- 状态转换图（语法图）是非常有用的设计工具
- 语法分析器和词法分析器的状态转换图不同
  - 每个非终结符对应一个状态转换图，边上的标记是记号和非终结符
  - 记号上的转换意味着如果该记号是下一个输入符号，就应进行转换
  - 非终结符A上的转换是对与A对应的过程的调用

4.4.2 语法图和递归子程序法

- 从文法构造语法图，对每个非终结符A执行如下操作
  - 创建一个开始状态和一个终止状态（返回状态）
  - 对每个产生式$A \rightarrow X_1 X_2 \ \ldots X_n$，创建一条从开始状态到终止状态的路径，边上的标记分别为X_1，X_2，…，X_n

例4.15 简单表达式文法的语法图



E→TE‘

E'→+TE'|ε

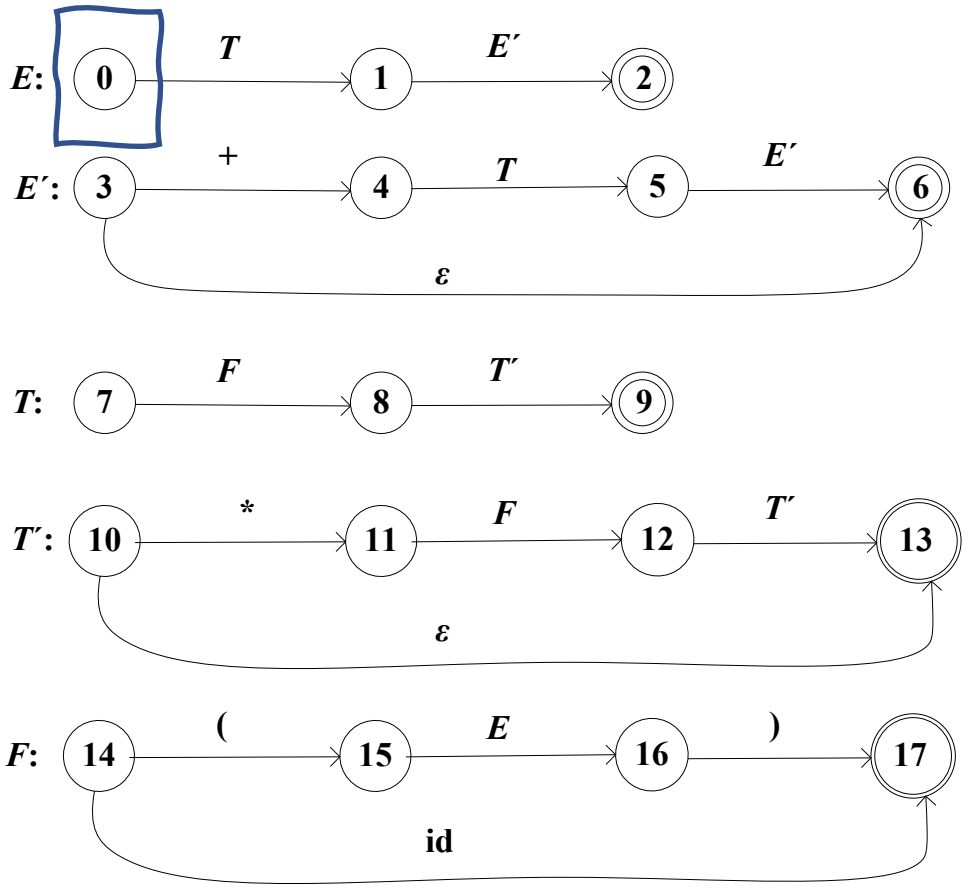
T→FT'

T'→*FT'|ε

F→(E)|id

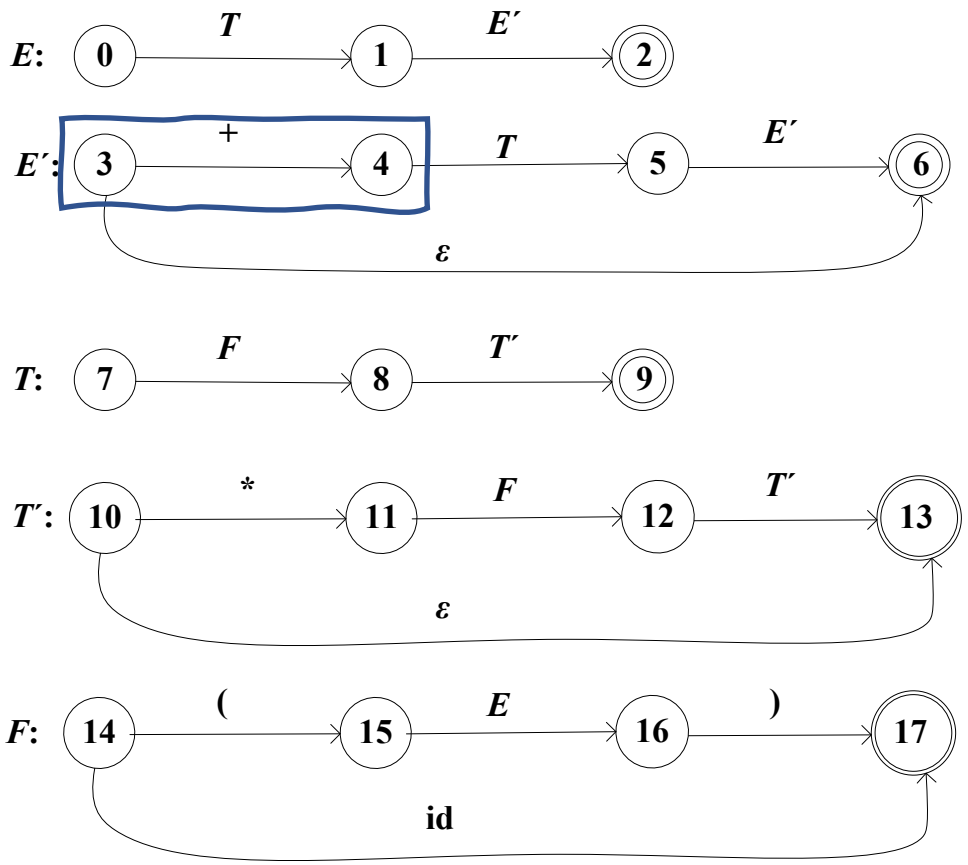
4.4.3基于语法图的语法分析器工作方式

- 初始时，分析器进入状态图的开始状态，输入指针指向输入符号串的第一个符号。



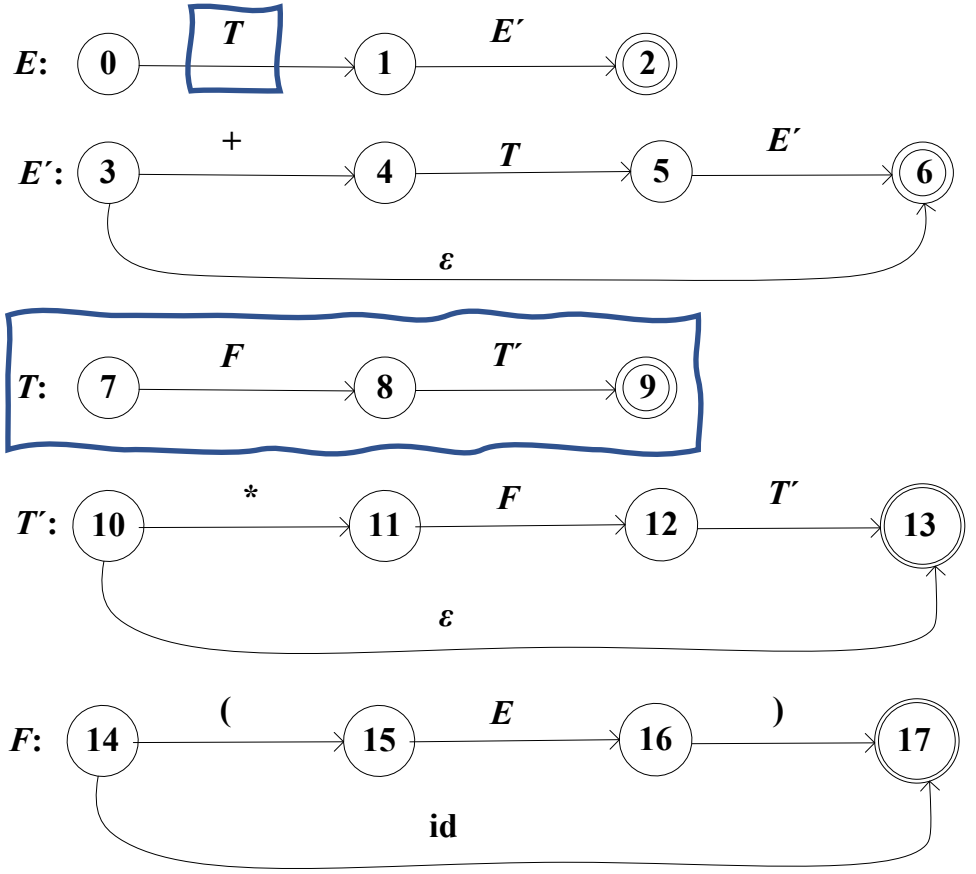
4.4.3基于语法图的语法分析器工作方式

- 如果经过一些动作后，它进入状态s，且从状态s到状态t的边上标记了终结符*a*，此时下一个输入符又正好是*a*，则分析器将输入指针向右移动一位，并进入状态t。



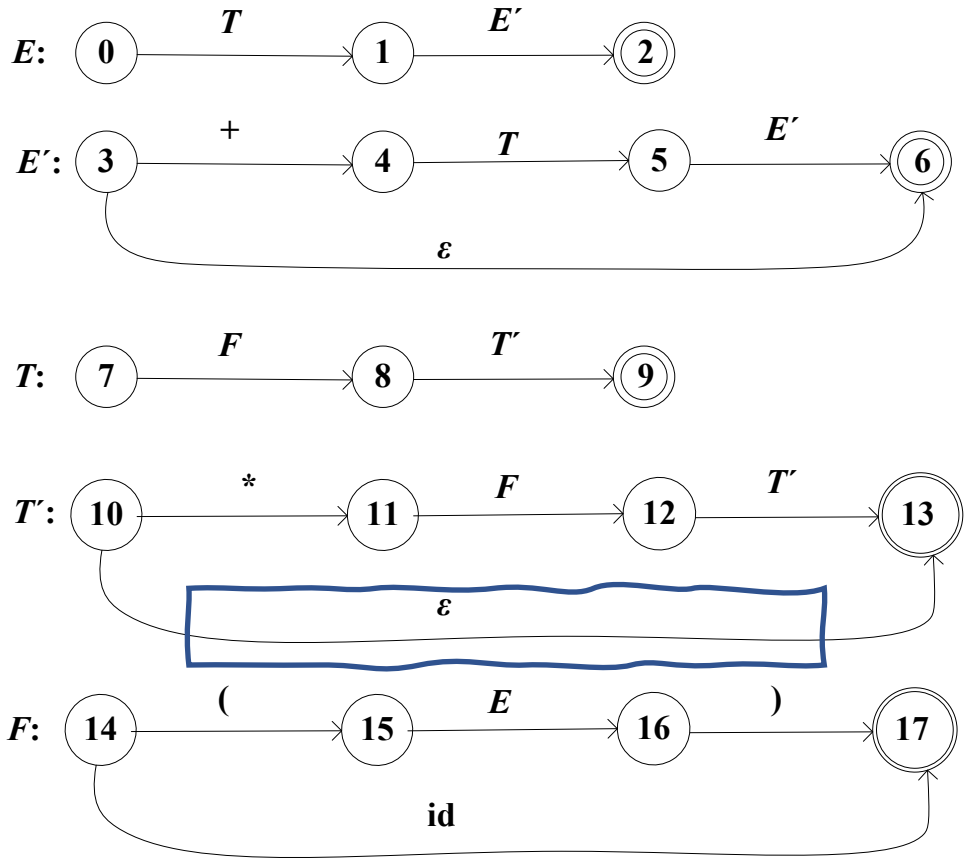
4.4.3基于语法图的语法分析器工作方式

- 另一方面，如果边上标记的是非终结符A，则分析器进入A的初始状态，但不移动输入指针。一旦到达A的终态，则立刻进入状态t，事实上，分析器从状态s转移到状态t时，它已经从输入符号串“读”了A （调用A对应的过程）。



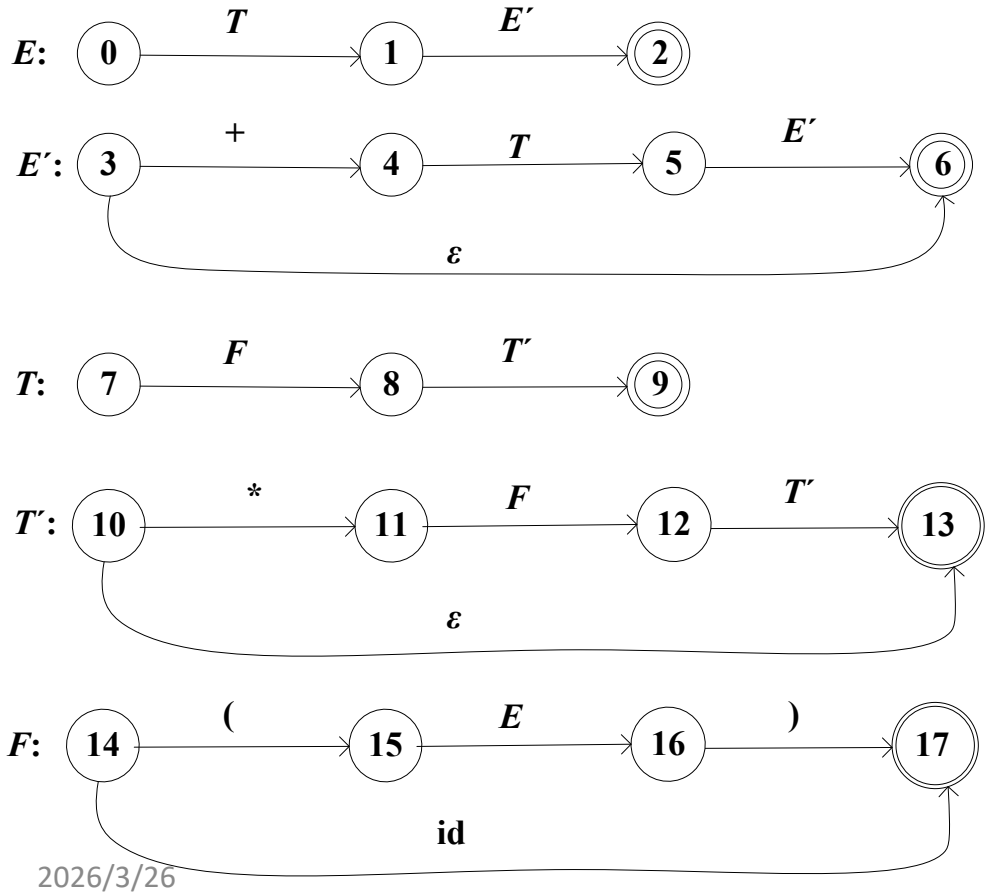
4.4.3基于语法图的语法分析器工作方式

- 如果从s到t有一条标记为ε的边，那么分析器从状态s直接进入状态t而不移动输入指针。



4.4.4 语法图的化简与实现

(1) 左因子提取 将形如$A \rightarrow YX|YZ$的产生式替换为$A \rightarrow Y(X|Z)$;
(2) 右因子提取 将形如$A \rightarrow YX|ZX$的产生式替换为$A \rightarrow (Y|Z)X$;
(3) 尾递归消除 将形如$X \rightarrow YX|Z$的产生式替换为$X \rightarrow Y^*Z$。



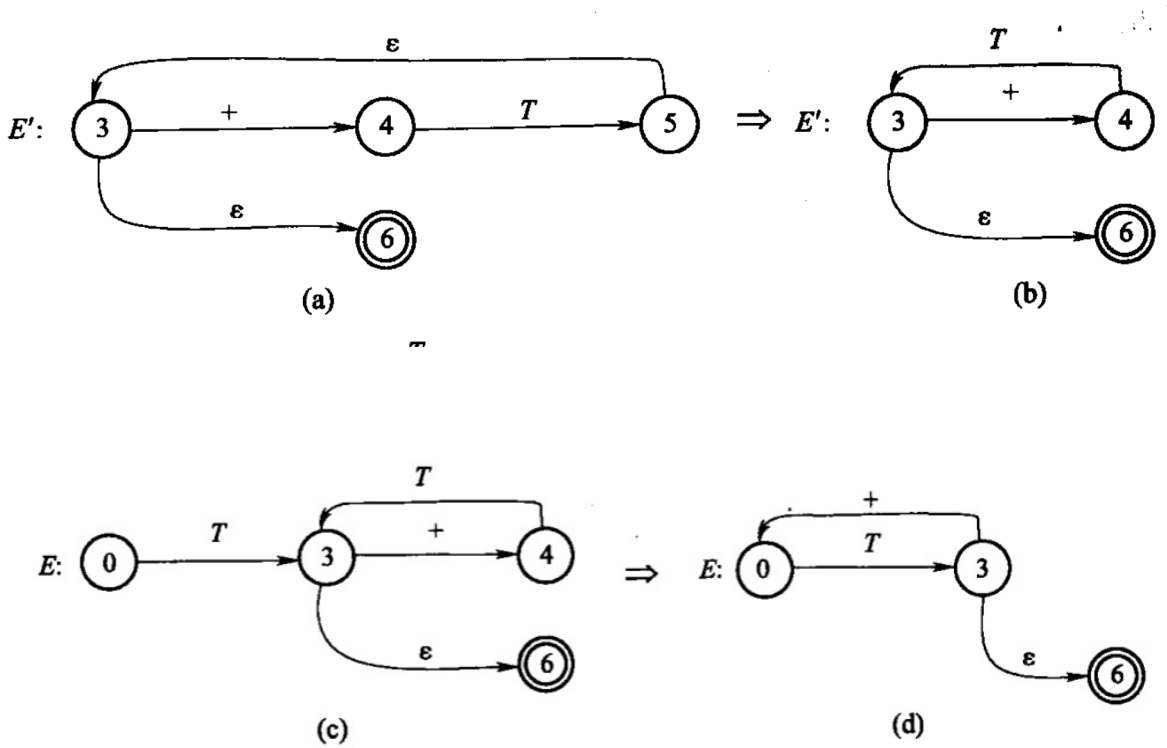
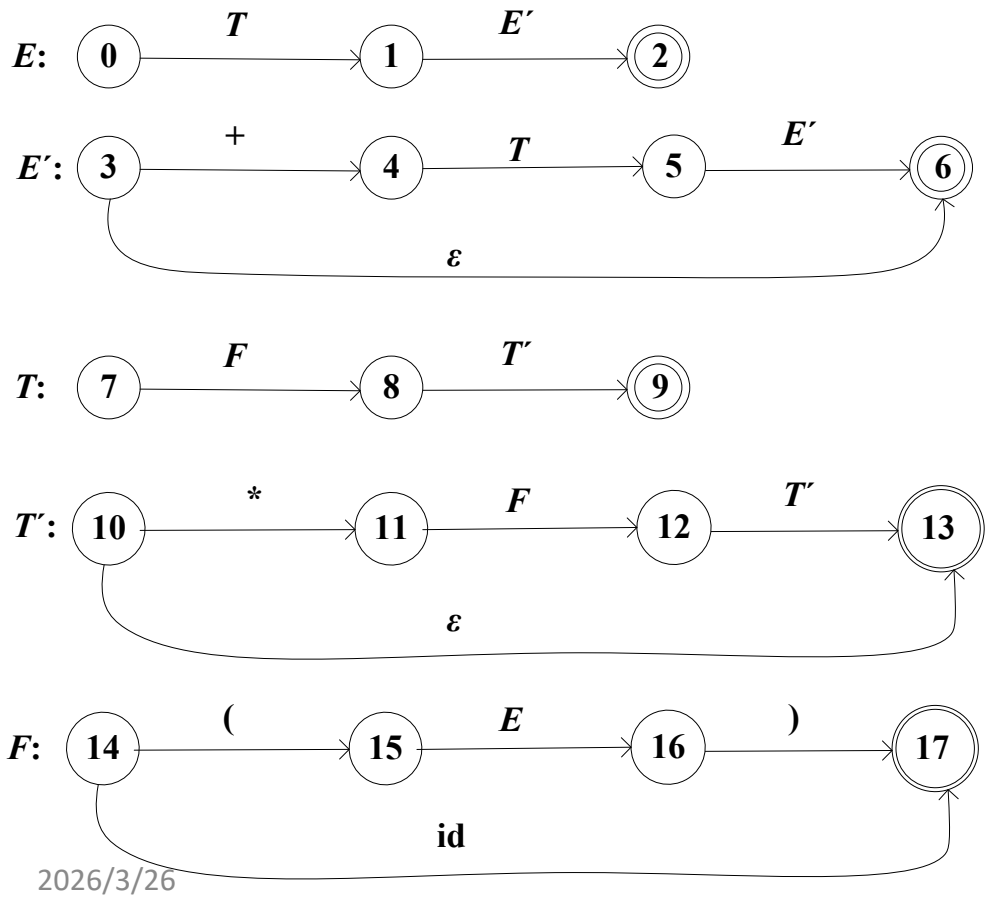


图4.6算术表达式的简化语法图

4.4.4 语法图的化简与实现

(1) 左因子提取 将形如$A \rightarrow YX|YZ$的产生式替换为$A \rightarrow Y(X|Z)$;
(2) 右因子提取 将形如$A \rightarrow YX|ZX$的产生式替换为$A \rightarrow (Y|Z)X$;
(3) 尾递归消除 将形如$X \rightarrow YX|Z$的产生式替换为$X \rightarrow Y^*Z$。



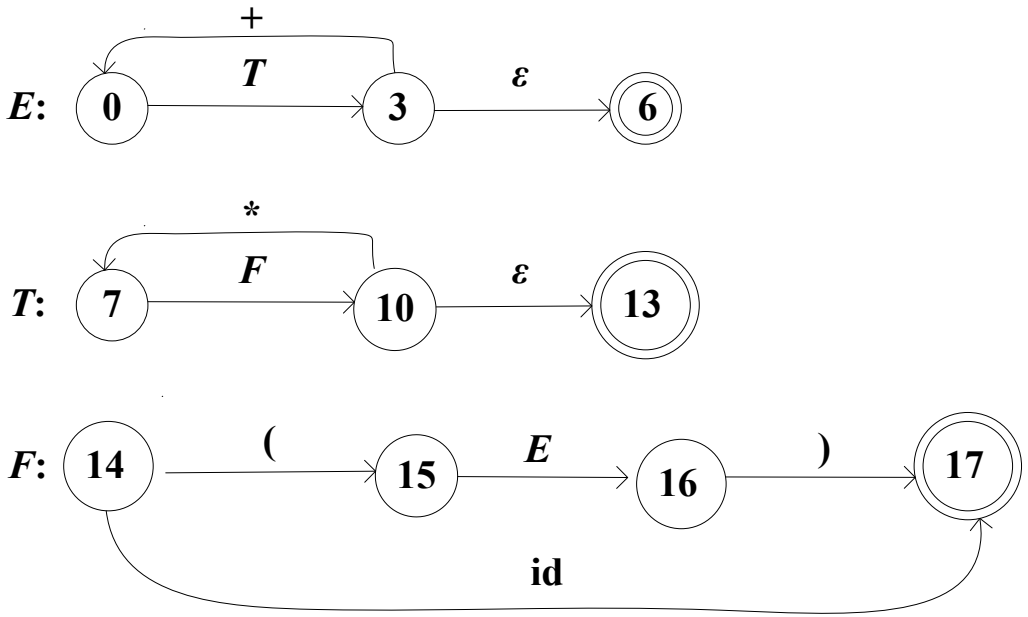
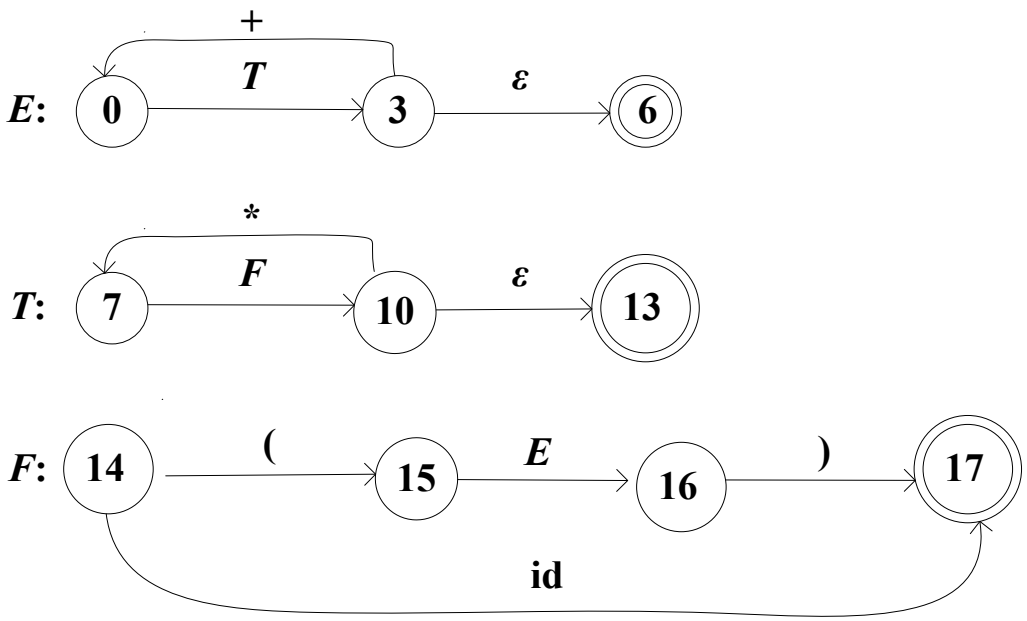


图4.6算术表达式的简化语法图

例4.16 简单算术表达式的语法分析器

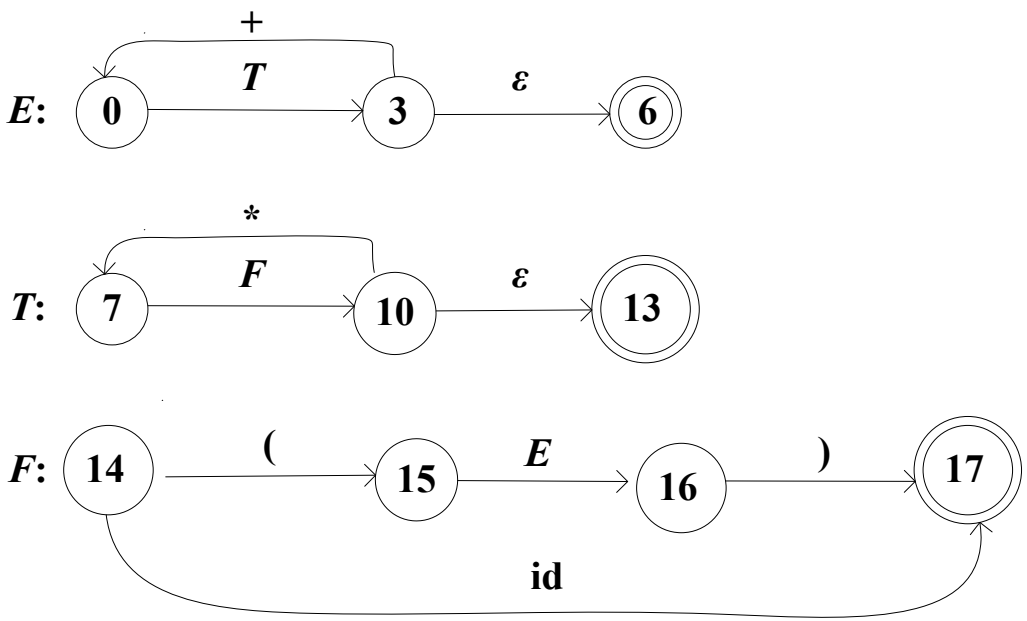
- E 的子程序(E→T(+T)*)

```
procedure E;
 begin
   T;                                T的过程调用
   while lookhead='+' do
    begin                  当前符号等于+时
      match(‘+’);                  处理终结符+
      T                  T的过程调用
    end
 end;                              lookhead:  当前符号
```



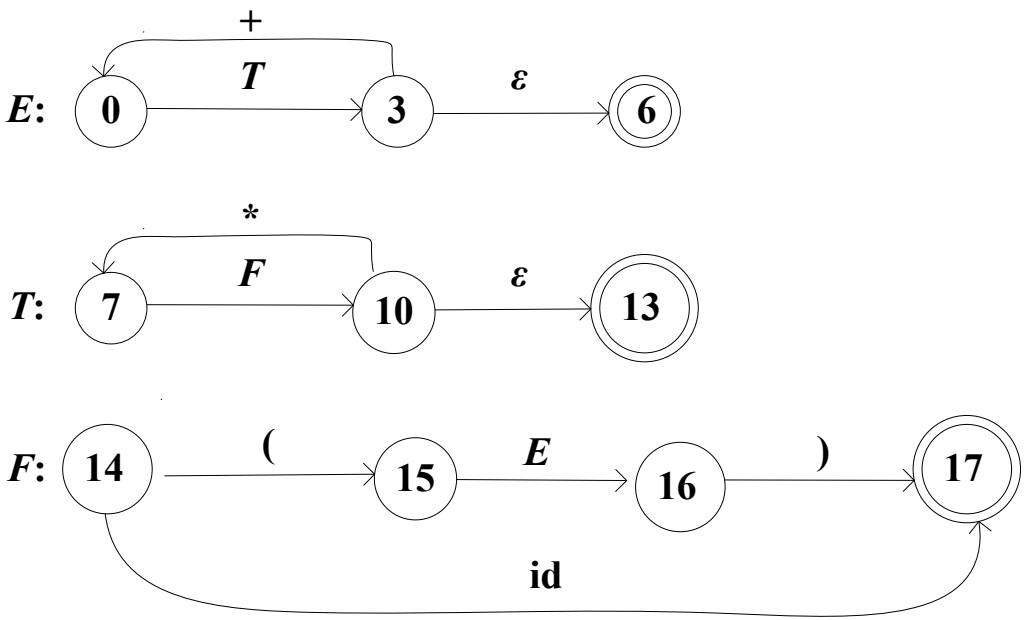
T 的子程序(T→F(*F)*)

```
procedure T;
  begin
     F;                          F的过程调用
     while lookhead='*' then
       begin                     当前符号等于*时
          match(‘*’);            处理终结符*
          F                      F的递归调用
       end
  end;
```



F 的子程序(F→(E)|id)

```
procedure F;
  begin
     if lookhead='(' then
        begin                    当前符号等于（
           match('(');           处理终结符（
           E;                    E 的递归调用
           match(')');           处理终结符)
        end
     else if lookhead=id then
           match(id) ;           处理终结符id
     else error                  出错处理
  end
```



主程序

```
begin
    lookhead:=nexttoken;
    E
end
```

调词法分析程序

E 的过程调用

服务子程序

```
procedure match(t:token);
 begin
    if lookhead=t  then
         lookhead:=nexttoken
    else error          出错处理程序
 end;
```

4.4.5 递归子程序法的实现步骤

1) 构造文法；

2) 改造文法：消除二义性、消除左递归、提取左因子；

3) 求每个候选式的FIRST集和语法变量的FOLLOW集；

4) 检查G是不是 $LL(1)$ 文法，若G不是 $LL(1)$文法，说明文法G的复杂性超过了自顶向下方法的分析能力，需要附加新的“信息”；

5) 按照$LL(1)$文法画语法图；

6) 化简语法图；

7) 按照语法图为每个语法变量设置一个子程序。

递归子程序法的优缺点分析

- 优点：
  1）直观、简单、可读性好
  2）便于扩充
- 缺点：
  1) 递归算法的实现效率低
  2) 处理能力相对有限
  3) 通用性差，难以自动生成
- 从递归子程序法及FIRST与FOLLOW集看如何进一步用好当前的输入符号？

本章小结

1. 自顶向下分析法和自底向上分析法分别寻找输入串的最左推导和最左归约
2. 自顶向下分析会遇到二义性问题、回溯问题、左递归引起的无穷推导问题，需对文法进行改造：消除二义性、消除左递归、提取公共左因子
3. *LL*(1)文法是一类可以进行确定分析的文法，利用FIRST集和FOLLOW集可以判定某个上下文无关文法是否为*LL*(1)文法

本章小结

4. *LL*(1)文法可以用*LL*(1)分析法进行分析。

5. 递归下降分析法根据各个候选式的结构为每个非终结符编写一个子程序。

6. 使用语法图可以方便地进行递归子程序的设计。